

Python Data Workflows

Practical Review Notes

This chapter is not about memorizing APIs. It begins by asking what shape the data has, then choosing the right tool: pair aligned lists with `zip`, keep positions with `enumerate`, count repeated categories with `Counter`, group records with `defaultdict(list)`, process FIFO tasks with `deque`, and stream values with a generator. Before moving to NumPy, these are the most useful ways to make everyday data processing concise and Pythonic.

- Python 3
- tutor style
- visual explanations
- FAQ included
- 6 core tools

00 Review Order and Decision Table

When you receive a batch of data, do not rush to write a loop. First ask, "What shape does this data have?" The answer usually points directly to the tool.

Data shape or task	Recommended tool
Two aligned lists, such as IDs and heart rates	<code>zip</code>
You need both the value and its position	<code>enumerate</code>
Count repeated categories, symptoms, or labels	<code>Counter</code>
Collect records by category, such as patient IDs by department	<code>defaultdict(list)</code>
Process tasks in first-in, first-out order	<code>deque</code>
Produce or clean data one item at a time	<code>generator + yield</code>

01 • START
zip
pair positions

02
Comprehension
filter + collect

03
enumerate
index + value

04
Counter
count categories

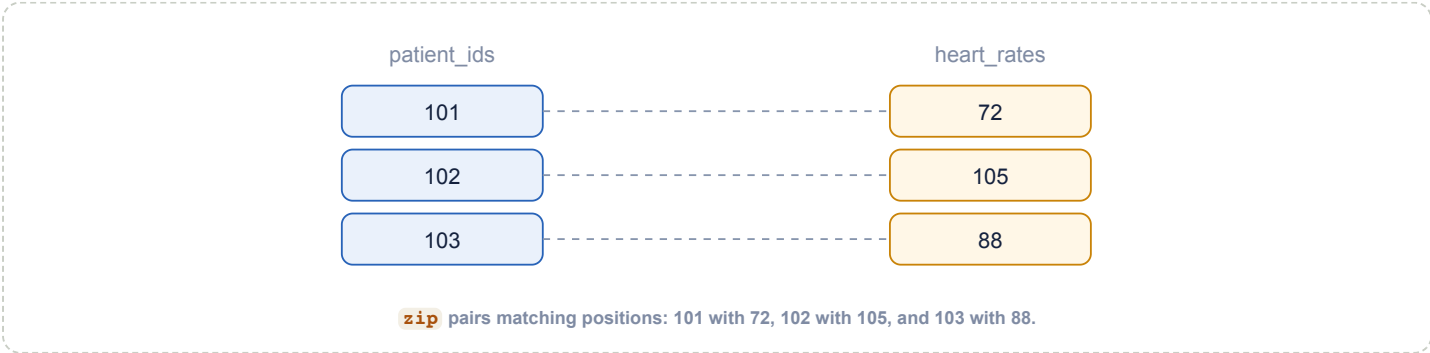
05
defaultdict
group records

06-07
deque + yield
queue + stream

These six tools are not isolated new ideas. `zip` and `enumerate` are richer forms of iteration; `Counter` and `defaultdict` package familiar counting and grouping templates; `deque` is the efficient queue from earlier work; and a generator is an iterator that does not store every result at once. This chapter turns patterns you already know into shorter, clearer, and more idiomatic code.

01 zip - Align Two Lists by Position

When the first item of one list corresponds to the first item of another, use `zip` to iterate through aligned pairs instead of managing indices manually.



```
for patient_id, rate in zip(patient_ids, heart_rates):
    print(patient_id, rate)
# 101 72
# 102 105
# 103 88
```

Q. Does zip check whether the two lists have the same length?

No. `zip` does not raise an error; it silently stops when the shorter list ends. With `ids=[101,102,103]` and `rates=[72,105]`, ID 103 is discarded without a warning. When every ID must match a measurement, check `len(ids) != len(rates)` first and raise an error instead of allowing silent truncation.

02 List Comprehensions - Filter and Collect

A list comprehension and a regular `for` loop can express exactly the same logic. Learn the loop clearly first, then compress it into a comprehension.

```
# Regular loop
result = []
for patient_id, rate in zip(patient_ids, heart_rates):
    if 60 <= rate <= 100:
        result.append(patient_id)

# List comprehension: the same operation
result = [
    patient_id
    for patient_id, rate in zip(patient_ids, heart_rates)
    if 60 <= rate <= 100
]
```

Read a comprehension in this order: "For every `patient_id, rate` pair, if `rate` is in the normal range, place `patient_id` in the result list." It is the three lines of a loop written in the order: what to collect, where to iterate, and which condition to keep.

Q. When should I use a regular loop, and when should I use a comprehension?

The logic is identical, so the choice is about readability. A regular loop is better for beginners, debugging, or multi-step work after filtering. A list comprehension is ideal for a simple "filter or transform, then collect" task that can be stated in one sentence. When both are possible, write the loop first to verify the logic, then compress it.

03 enumerate - Get the Position and the Value

When you need both "which value satisfies the condition" and "where it appears," use `enumerate`. Every iteration gives two items: the `index` and the actual value.

index

0

1

2

3

rate

72

105

88

112

(0,72)

(1,105)

(2,88)

(3,112)

`enumerate(rates) -> (0,72), (1,105), (2,88), (3,112)`

```

return [
    (index, rate)
    for index, rate in enumerate(rates)
    if rate > 100
]
# [(1, 105), (3, 112)]

```

Q. Why is the condition `if rate > 100`, not `if index > 100`?

This is the easiest detail to mix up: `index` is only the position 0, 1, 2, 3, while `rate` is the actual heart-rate value. A condition should usually test the real data, not its position, unless the question explicitly asks about positions such as "after the third item."

04

Counter - Count Category Frequencies

Counter is a dictionary tool designed specifically for counting. Conceptually, it is the official packaged version of the manual counting template you learned earlier.

["fever", "cough", "fever", "fatigue", "cough", "cough"]

cough: 3

fever: 2

fatigue: 1

Counter(symptoms) counts every category automatically.

Counter vs the equivalent manual pattern

counter

```

from collections import Counter
counts = Counter(symptoms)
# Counter({"cough": 3, "fever": 2, "fatigue": 1})

# Manual version: equivalent to Counter
def symptom_counts_manual(symptoms):
    counts = {}
    for symptom in symptoms:
        counts[symptom] = counts.get(symptom, 0) + 1
    return counts

```

Why learn the manual version too? Some problems require an extra condition while counting, such as counting only symptoms recorded after a certain date. In that case `Counter(symptoms)` may not be flexible enough, so you return to a loop with `counts.get(k, 0) + 1`. `Counter` is faster to write; the manual pattern is more adaptable. Know both.

05

defaultdict(list) - Group Records by Category

Use it when the goal is "one list for each category." The first time a category appears, it automatically creates an empty list, so you do not have to test whether the key exists.

records: [(Cardiology,101), (Neurology,102), (Cardiology,103)]

Cardiology

[101, 103]

Neurology

[102]

First "Neurology" visit: create [] automatically, then append(102).

```
from collections import defaultdict

groups = defaultdict(list)
for department, patient_id in records:
    groups[department].append(patient_id)
# First access -> create [] automatically -> append
```

Q. Why is `defaultdict([])` wrong?

`defaultdict` expects a function that creates a default value, not the default object itself. `list` is a function; calling `list()` creates a new empty list. By contrast, `[]` is one specific list object. If one list were shared by every key, the groups would contaminate one another. Pass `list` without parentheses so every missing key receives its own new list.

Q. How is this better than manually checking whether a key exists?

The manual pattern needs `if department not in groups: groups[department] = []` before `append`. `defaultdict(list)` builds that check into `groups[department].append(patient_id)`. It is shorter, clearer, and the standard way to express grouping.

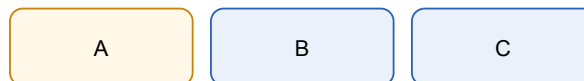
06

deque - An Efficient FIFO Queue

A queue is FIFO: first in, first out. `deque` is designed for efficient operations at both ends. Removing from the left with `popleft()` is $O(1)$, while `list.pop(0)` is $O(n)$.

popleft() $O(1)$

append() joins the queue



A is served first.

Before [A,B,C] -> popleft() -> after [B,C]

deque - process a queue

 $O(1)$

```
from collections import deque

queue = deque(patient_queue)
served = []
while queue:
    patient = queue.popleft()
    served.append(patient)
return served

# Why not list.pop(0)?
# items.pop(0)      # O(n): shift every later element left
# queue.popleft()   # O(1): deque is built for two-ended operations
```

Complexity analysis from earlier work: this is a live example of "data structure determines complexity." Both methods remove the first item, but a list needs $O(n)$ shifting while a deque needs $O(1)$ work because it uses a double-ended structure. Small inputs may feel the same; at scale, the advantage is obvious.

07

Generators and yield - Produce Data One Item at a Time

When data does not need to be stored in a complete list, use a generator. It yields one value at a time, computes only what is requested, and does not occupy a large block of memory in advance.

readings = [72, None, 80, None, 76]

72

None

80

None

76

yield 72 -> pause -> yield 80 -> pause -> yield 76

list(...) -> [72, 80, 76]

Each valid value is yielded once; the function pauses and resumes at the next item.

generator - filter invalid readings lazily

yield

```
def valid_sensor_values(readings):
    for reading in readings:
        if reading is not None:
            yield reading

readings = [72, None, 80, None, 76]
print(list(valid_sensor_values(readings)))
# [72, 80, 76]
```

Q. Why use `if reading is not None` instead of `if reading`?

Because 0 is also False in Python. If 0 is a valid sensor reading, as in `[0, 72, None]`, `if reading` incorrectly discards that valid 0. An explicit `is not None` removes only the true missing value.

Q. What happens if `return reading` replaces `yield reading`?

`return` ends the entire function immediately, so only the first valid value is obtained and every later value is lost. `yield` means "hand over one value and pause." The function keeps its state and continues from that point on the next iteration. This is the essential difference between a generator and a regular function.

Q. Why wrap the result in `list(...)` at the end?

A generator is not itself a list; it is an object that produces values one at a time, so printing it directly does not show its contents. `list(generator)` consumes it once and collects the values into a real list for inspection or return. In genuinely large pipelines, however, you often do not convert it to a list, because lazy processing is what saves memory.

08

Bug and FAQ Summary

Incorrect pattern	Why it fails	Correct approach
zip lists of unequal length	Silent truncation discards items from the longer list.	Check <code>len(a) != len(b)</code> first.
if index > 100	index is the position, not the data value.	if rate > 100
defaultdict([])	[] is an object, not a factory function; keys would not receive fresh lists.	defaultdict(list)
Manually check every key	It works, but grouping is exactly what defaultdict is meant to express.	defaultdict(list) handles it in one line.
Use pop(0) on a large list	Every removal shifts all later elements, so it is O(n).	deque.popleft(), O(1)
if reading: to remove missing values	It also removes the valid value 0.	if reading is not None:
return reading inside a generator	It ends the function after the first value.	yield reading

High-frequency FAQ

Q. Are Python None and NumPy np.nan the same?

No. `None` usually means "no value" in ordinary Python, while `np.nan` is the missing-value marker commonly used in NumPy and Pandas numeric data. This chapter uses `reading is not None`; after moving to NumPy, use `np.isnan(array)`. They represent different objects and must not be tested in the same way.

Q. What is the difference between Counter and a regular dict?

`Counter` is a dictionary tool dedicated to counting and is more direct than manually writing `.get(..., 0) + 1`. Still understand the manual loop, because some questions require extra conditions inside the counting process, such as filtering by date, where a plain loop is more flexible.

Return to the whole chapter: all six tools follow the same idea - inspect the data shape first, then choose the container or syntax that fits it. This is a turning point from "code that works" toward idiomatic Python: the logic is the same, but the correct tool makes it shorter, faster, and easier for other people to understand.

09 Exercises and Chapter Checklist

Practice problems

- E Find patients with abnormal temperatures:** `ids=[1,2,3,4]`, `temperatures=[36.8,38.6,37.2,39.1]`. Return IDs whose temperature is strictly above 38.0. Expected: `[2,4]`. Practice `zip` plus a list comprehension.
- E Find positions and values with low oxygen:** `oxygen_levels=[98,92,96,89,97]`. Return each `(index, value)` below 95. Expected: `[(1,92),(3,89)]`. Practice `enumerate`.
- E Count diagnosis labels:** `["flu","covid","flu","asthma","flu","covid"] -> {"flu":3,"covid":2,"asthma":1}`. Practice `Counter` and the equivalent `dict.get` pattern.
- M Group patient IDs by department:** use the `records` example above. Practice `defaultdict(list)`.
- M Simulate a triage queue:** serve patients in arrival order and return the served order. Practice `deque`, `while`, and `popleft()`.
- M Clean sensor readings lazily:** `[0,None,73,None,81] -> [0,73,81]`. Practice `yield` plus `is not None`, and make sure the valid 0 is preserved.

• Chapter checklist

`zip` pairs items at matching positions

unequal `zip` lengths cause silent truncation

a comprehension can express a simple filter and collect

`enumerate` returns `(index, value)`

`Counter` counts frequencies

grouping suggests `defaultdict(list)`

FIFO queues use `deque.popleft()`

`yield` produces values one at a time

test missing values with `is not None`, not `if value`

classify the task as pair, count, group, queue, or stream

After every exercise: your instinct for handling tabular data is now taking shape. These tools will reappear when you use Pandas `groupby`, clean CSV or JSON files, and prepare training data for models. They are not isolated syntax tricks; they are the everyday toolbox for turning real records into reliable processing pipelines.